



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Facultad de Minas

**Departamento de Ciencias de la
Computación y de la Decisión**

Proyecto Terraform (20%)

Servicios en la Nube 2026-01

Número de Integrantes: máximo 3.

Fecha de entrega: Lunes 22 de junio (Provisional).

Herramientas obligatorias: Terraform & Google Cloud Platform (GCP)

Escenario

Trabajas como Ingeniero de Cloud/DevOps en una startup de rápido crecimiento. El equipo de desarrollo está por lanzar una actualización crítica para la plataforma web. Para evitar pérdidas millonarias si algo sale mal, la dirección de la empresa te ha pedido diseñar e implementar una arquitectura que sea **altamente flexible en el manejo de tráfico de red**.

Tu objetivo es entregar la infraestructura completamente automatizada como código (IaC), de modo que el equipo de operaciones pueda alterar el comportamiento del tráfico modificando únicamente un archivo de variables, sin necesidad de tocar la consola web de la nube ni reescribir el código de la arquitectura.

Desafío

Debes diseñar e investigar una solución en la nube que cumpla estrictamente con el siguiente comportamiento:

- **El Servicio Principal:** Debe existir un destino que represente la aplicación en producción. Al ingresar, debe mostrar un mensaje claro en el navegador: "**Bienvenido al Servicio Principal - Versión Producción**".
- **El Servicio de Contingencia:** Debe existir un destino alternativo que actúe como página de error o mantenimiento. Al ingresar, debe mostrar en el navegador: "**Error 503 - Sitio en Mantenimiento Programado**".
- **El Punto de Entrada Único:** Los usuarios en internet solo tendrán acceso a una única dirección IP pública. No deben conocer las IPs internas de los servidores de fondo.
- **Control de Tráfico por Variables:** A través de tu archivo de configuración de variables de Terraform (`variables.tf` o `terraform.tfvars`), se debe poder declarar cuantitativamente



(por ejemplo, mediante porcentajes o pesos) cómo se va a distribuir el tráfico que llega a esa única IP pública hacia ambos servicios.

- **Restricción Crítica de Arquitectura (Aislamiento de Fallos):** El *Servicio Principal* y el *Servicio de Contingencia* deben residir en recursos de cómputo (instancias/servidores) **completamente independientes**. Bajo ninguna circunstancia pueden compartir la misma máquina virtual. Recuerda que el propósito central de la contingencia es garantizar la resiliencia: si la infraestructura del servicio principal falla o se destruye, el servicio de contingencia debe seguir operando con total normalidad para responder a los usuarios.

Condiciones del Entorno

- **Plataforma Obligatoria:** Google Cloud Platform (GCP).
- **Presupuesto:** El proyecto debe ejecutarse optimizando los costos al máximo utilizando recursos pequeños. Recuerda que si abres una cuenta nueva en GCP, la plataforma te otorga **\$300 USD en créditos gratuitos**, los cuales son más que suficientes para este reto si eliges bien el tamaño de tus recursos.
- **Automatización Absoluta:** Todo el proceso (redes, servidores, páginas web internas y balanceo) debe desplegarse con un solo comando: **terraform apply**. **No se permite la configuración manual** de los servidores ni de las páginas web a través de SSH o de la consola web tras el despliegue.

Criterios de Aceptación (Escenarios de Evaluación)

El código de Terraform será probado y evaluado en tres escenarios distintos, cambiando **únicamente** los valores de tus variables de porcentaje/peso de tráfico:

- **Escenario 1 (Producción Activa):** Al configurar las variables para producción total (100% - 0%), todas las visitas a la IP pública deben ver el *Servicio Principal*.
- **Escenario 2 (Mantenimiento Total):** Al cambiar las variables a modo contingencia (0% - 100%), todas las visitas a la misma IP pública deben ser redirigidas a la *Página de Error*.
- **Escenario 3:** Al configurar las variables en un balance equitativo (50% - 50%), el sistema debe distribuir las peticiones de forma que un usuario (o peticiones consecutivas) vea alternadamente ambos servicios al actualizar el navegador.



Configuración de Accesos e IAM (Obligatorio para la revisión)

Para que el profesor (asistido por herramientas de IA) pueda auditar, simular los escenarios y desplegar la infraestructura directamente en tu entorno (utilizando tus créditos de GCP), deberás configurar el acceso en tu consola antes de la fecha de entrega:

1. Ve al menú de **IAM y administración > IAM** en tu consola de GCP.
2. Añade un nuevo miembro utilizando el correo electrónico del profesor: vdrestrepot@unal.edu.co.
3. Asígnale el rol de **Proyecto -> Editor** ([roles/editor](#)). Este rol es indispensable para que las herramientas de automatización puedan interactuar con tu proyecto.
4. Asegúrate de que el ID exacto de tu proyecto de GCP esté parametrizado como una variable dentro de tu código. El repositorio debe ser capaz de ejecutarse en tu proyecto sin necesidad de que el profesor modifique tus archivos [.tf](#).

> **Nota de seguridad:** Una vez publicada la nota final del proyecto, podrás revocar este acceso de inmediato eliminando el correo del profesor de tu lista de IAM.

Entregables

La entrega se realizará a través de correo, enviando un mensaje con:

1. **Enlace al Repositorio:** Link público (o con acceso para el profesor) a tu código en GitHub o GitLab. El código debe estar limpio y formateado con el comando [terraform fmt](#).
2. **Documentación (README.md y AGENTS.md):** Un manual breve dentro del repositorio que explique qué variables se deben modificar para activar cada uno de los 3 escenarios de evaluación, acompañado de las evidencias (capturas de pantalla o logs de peticiones) correspondientes. También un documento donde se le explique a un LLM cómo leer y entender el proyecto. Esta documentación también puede estar en el repositorio.
3. **Cierre de Proyecto:** Una captura de pantalla final que evidencie la ejecución exitosa del comando [terraform destroy](#) para asegurar que tu entorno quedó limpio y no siga consumiendo tus créditos. Recuerda que es el profesor quien va a desplegar los recursos desde tu repositorio, de manera que no necesitas entregar nada desplegado.

¡NOTA CRÍTICA SOBRE EL ESTADO DE TERRAFORM (REQUISITO DE REVISIÓN)! Al realizar las pruebas de tu código localmente, Terraform genera un archivo de estado interno ([terraform.tfstate](#)). Es **estrictamente obligatorio** que ejecuten el comando [terraform destroy](#) en



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Facultad de Minas

**Departamento de Ciencias de la
Computación y de la Decisión**

sus terminales antes de dar por terminado el taller. Su consola de GCP debe quedar completamente vacía de los recursos creados por este proyecto.

Recuerda que el profesor ejecutará el despliegue desde cero utilizando únicamente el código de tu repositorio. Si dejas recursos activos en tu cuenta, el script de revisión automatizada no reconocerá el estado previo de tus máquinas locales, intentará duplicar los recursos y la plataforma de GCP arrojará un error inmediato de conflicto por nombres duplicados, lo cual invalidará la revisión de tu entrega. **Si esto sucede, automáticamente tendrás un cero (0.0) en tu nota.**

Asunto Obligatorio del Correo: Para facilitar la clasificación y el procesamiento automatizado de las entregas, el correo electrónico debe enviarse **estrictamente** con el siguiente formato en el asunto: [Servicios Nube 2026-01] Proyecto Terraform - Grupo [Número de Grupo] *Ejemplo:* [Servicios Nube 2026-01] Proyecto Terraform - Grupo 3

Correos que no cumplan con esta nomenclatura exacta en el asunto podrían quedar fuera del filtro de revisión inicial.

¡Mucho éxito! Recuerda que la clave del proyecto está en investigar la documentación oficial de Terraform para GCP sobre la gestión avanzada de tráfico en servicios de balanceo.